Nonparametric Regression

Lecture Notes

Giorgio Gambosi
Master's Degree in Computer Science
University of Rome "Tor Vergata"

A.Y. 2026–2027

Contents

1 Parametric and Nonparametric Models 1
2 The Equivalent Kernel 1
3 Kernel Regression 2
3.1 The Nadaraya–Watson Estimator . . . 2
3.2 k-Nearest Neighbors . . . 3
3.3 Bandwidth Selection . . . 4
4 Locally Weighted Regression 4
4.1 Local Linear Fits . . . 4
4.2 Local Logistic Regression . . . 5
5 Gaussian Processes 5
5.1 From a Prior on Parameters to a Prior on Functions . . . 5
5.2 Definition . . . 7
5.3 Kernels . . . 7
6 Gaussian Process Regression 8
6.1 Noise-Free Observations . . . 8
6.2 Noisy Observations . . . 10
7 Kernels Across Methods 11
8 Choosing the Kernel Parameters 11

1 Parametric and Nonparametric Models

The models of the notes on linear regression are parametric: a functional form, such as $h(\mathbf{x}; \overline{\mathbf{w}}) = \overline{\mathbf{w}}^T \overline{\mathbf{x}}$ or a polynomial of degree M, is fixed before seeing the data, its parameters are estimated from the training set, and the data play no further role in prediction, which only uses the fitted parameter vector. This has real advantages: the model is compact, prediction at a new point costs one dot product, and each coefficient has a direct meaning. The limitation is structural: the functional form encodes an assumption about the shape of the regression function, and when the assumption is wrong no amount of data removes the resulting bias, as for a linear model fitted to a quadratic signal, which underfits whatever the size of the training set.

Nonparametric methods do not fix a functional form in advance, and let the data determine the shape of the prediction locally, around each query point. Their number of parameters is not fixed independently of n: the model grows in complexity as data arrive. The methods discussed here share a common mechanism, in which the prediction at a query point $\mathbf{x}$ aggregates the targets of the training items close to $\mathbf{x}$, with weights decreasing with the distance. Fully Bayesian linear

regression reveals that this structure is already implicit in a parametric model (Section 2); kernel regression makes the weighting explicit (Section 3); locally weighted regression replaces the local average with a local linear fit, and extends to classification (Section 4). Gaussian processes, finally, place a prior directly on functions, and obtain the same kind of predictor from Bayesian inference (Sections 5–8).

2 The Equivalent Kernel

In fully Bayesian linear regression, with prior $\mathcal{N}(\overline{\mathbf{w}} \mid \mathbf{0}, \alpha^{-1}\mathbf{I})$ and Gaussian noise with precision β, the predictive distribution is $\mathcal{N}(t \mid m(\mathbf{x}), \sigma^2(\mathbf{x}))$, with

$$m(\mathbf{x}) = \beta\, \overline{\mathbf{x}}^T \mathbf{\Sigma} \overline{\mathbf{X}}^T \mathbf{t}, \qquad \sigma^2(\mathbf{x}) = \frac{1}{\beta} + \overline{\mathbf{x}}^T \mathbf{\Sigma} \overline{\mathbf{x}}, \qquad \mathbf{\Sigma} = \left(\alpha\mathbf{I} + \beta \overline{\mathbf{X}}^T \overline{\mathbf{X}}\right)^{-1}, \tag{1}$$

as derived in the notes on linear regression. Since $\overline{\mathbf{X}}^T \mathbf{t} = \sum_{i=1}^n \overline{\mathbf{x}}_i\, t_i$, the predictive mean can be written as

$$m(\mathbf{x}) = \beta\, \overline{\mathbf{x}}^T \mathbf{\Sigma} \sum_{i=1}^n \overline{\mathbf{x}}_i\, t_i = \sum_{i=1}^n \kappa(\mathbf{x}, \mathbf{x}_i)\, t_i, \qquad \kappa(\mathbf{x}, \mathbf{x}') = \beta\, \overline{\mathbf{x}}^T \mathbf{\Sigma}\, \overline{\mathbf{x}}', \tag{2}$$

or $\kappa(\mathbf{x}, \mathbf{x}') = \beta\, \boldsymbol{\phi}(\mathbf{x})^T \mathbf{\Sigma}\, \boldsymbol{\phi}(\mathbf{x}')$ when basis functions are used. The prediction is a weighted sum of the training targets, in which no parameter vector appears: the weight of t_i depends on the query $\mathbf{x}$ and on the training input $\mathbf{x}_i$ through the **equivalent kernel** κ, which measures how strongly the targets at two points depend on each other.

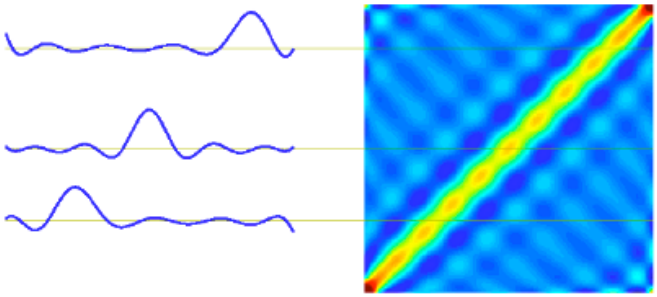


Figure 1: The equivalent kernel for Gaussian basis functions. Left: $\kappa(x, x_i)$ as a function of x, for three values of x_i. Right: κ on the plane (x, x_i). The largest weights are assigned to the training points closest to the query.

The equivalent kernel is *localized* (Figure 1): the prediction at $\mathbf{x}$ gives the largest weights to the targets of the training items close to $\mathbf{x}$. Localization is not imposed on the model, but results from averaging over $\overline{\mathbf{w}}$, and holds for different choices of the basis functions, with different tail behaviors (Figure 2). This suggests a direct approach: instead of choosing basis functions that implicitly produce a localized kernel, define a kernel on pairs of points and use it directly to weight the training targets.

3 Kernel Regression

3.1 The Nadaraya–Watson Estimator

Kernel regression predicts the target at a query point $\mathbf{x}$ by averaging the targets of the nearby training items, with larger weights for closer items. Closeness is measured by a **kernel** $\kappa_s(\mathbf{u})$, a non-negative function of the difference $\mathbf{u}$ between two points, non-negligible only within a window of

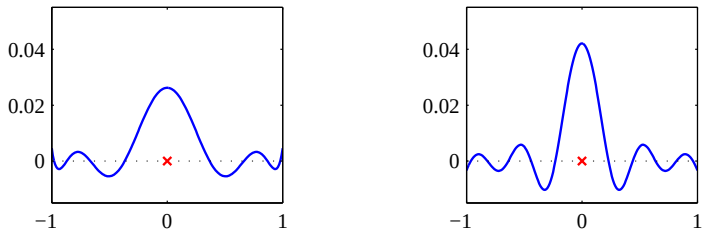


Figure 2: The equivalent kernel $\kappa(0, x)$ for polynomial (left) and Gaussian (right) basis functions: both are localized, with different tails.

radius about s around the origin. The **bandwidth** s sets the width of the neighborhood: a wide kernel averages over many items and produces a smooth prediction, a narrow one averages over few items and follows local variations. A common choice is the **Gaussian** kernel $\kappa_s(\mathbf{u}) \propto e^{-\|\mathbf{u}\|^2/2s^2}$, whose weight decays exponentially with the squared distance (Figure 3).

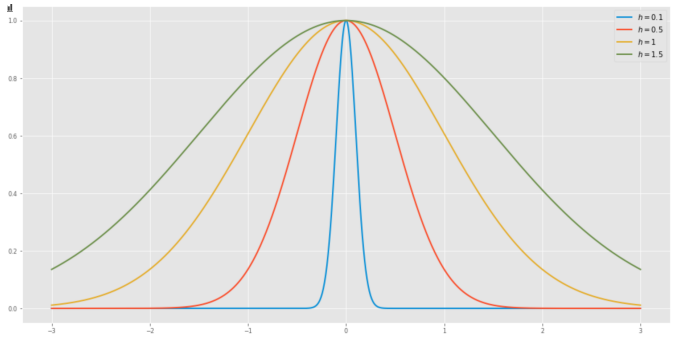


Figure 3: The Gaussian kernel for $d = 1$ and three values of the bandwidth: a larger bandwidth gives a wider and flatter kernel, which averages over a larger neighborhood.

Under the quadratic loss the optimal predictor is the conditional mean,

$$\mathbb{E}\left[t \mid \mathbf{x}\right] \;=\; \int t\, p(t \mid \mathbf{x})\, dt \;=\; \frac{\int t\, p(\mathbf{x}, t)\, dt}{p(\mathbf{x})} \;=\; \frac{\int t\, p(\mathbf{x}, t)\, dt}{\int p(\mathbf{x}, t)\, dt}, \tag{3}$$

in which the unknown joint density gives both numerator and denominator. It can be approximated without assuming a parametric form, by **kernel density estimation**, which places a kernel bump on each training pair and averages them:

$$p(\mathbf{x}, t) \;\approx\; \frac{1}{n} \sum_{i=1}^{n} \kappa_s(\mathbf{x} - \mathbf{x}_i)\, \kappa_s(t - t_i). \tag{4}$$

Substituting, and exchanging sums and integrals,

$$h(\mathbf{x}) \;=\; \frac{\sum_{i=1}^{n} \kappa_s(\mathbf{x} - \mathbf{x}_i) \int t\, \kappa_s(t - t_i)\, dt}{\sum_{i=1}^{n} \kappa_s(\mathbf{x} - \mathbf{x}_i) \int \kappa_s(t - t_i)\, dt}. \tag{5}$$

When κ_s is a probability density with zero mean, $\int \kappa_s(t - t_i)\, dt = 1$ and $\int t\, \kappa_s(t - t_i)\, dt = t_i$, and the prediction becomes the **Nadaraya–Watson estimator**

$$h(\mathbf{x}) \;=\; \sum_{i=1}^{n} w_i(\mathbf{x})\, t_i, \qquad w_i(\mathbf{x}) \;=\; \frac{\kappa_s(\mathbf{x} - \mathbf{x}_i)}{\sum_{j=1}^{n} \kappa_s(\mathbf{x} - \mathbf{x}_j)}, \tag{6}$$

a convex combination of the training targets, in which the items close to $\mathbf{x}$ dominate and the distant ones receive negligible weight. With basis functions, $\mathbf{x} - \mathbf{x}_i$ is replaced by $\boldsymbol{\phi}(\mathbf{x}) - \boldsymbol{\phi}(\mathbf{x}_i)$.

3.2 k-Nearest Neighbors

k-nearest neighbor regression is the special case of a uniform kernel restricted to the k training items $\mathbf{x}_{(1)}, \dots, \mathbf{x}_{(k)}$ closest to the query:

$$h(\mathbf{x}) \;=\; \frac{1}{k} \sum_{i=1}^{k} t_{(i)}. \tag{7}$$

The number k controls the bias–variance trade-off: a small k follows local variations but amplifies the noise, a large k reduces the noise but introduces bias where the regression function varies. Unlike Nadaraya–Watson, which uses the same bandwidth everywhere, k-NN adapts the width of the neighborhood to the local density of the data: in sparse regions the k nearest neighbors are far from $\mathbf{x}$, and the window is wide, while in dense regions it is narrow.

Example: adaptive neighborhoods

Consider a one-dimensional dataset in which 90% of the inputs lie in $[0, 1]$ and the remaining 10% are scattered in $[5, 10]$. A fixed bandwidth suited to the dense region covers very few points in the sparse one, where the Nadaraya–Watson estimate is noisy; a bandwidth suited to the sparse region over-smooths the dense one. k-NN uses a narrow window in $[0, 1]$ and a wide one in $[5, 10]$, since the k-th nearest neighbor is farther away there, allocating smoothing according to the availability of data.

3.3 Bandwidth Selection

The bandwidth is the most important hyperparameter of kernel regression: too small, and the estimator follows the noise; too large, and it over-smooths and misses real structure. A data-driven choice is given by leave-one-out cross-validation,[1] which chooses the bandwidth minimizing

$$\mathrm{CV}(s) \;=\; \frac{1}{n} \sum_{i=1}^{n} \big(t_i - h_{-i}(\mathbf{x}_i)\big)^2, \tag{8}$$

where h_{-i} is the estimator computed without the item $(\mathbf{x}_i, t_i)$; K-fold cross-validation is preferable for large n. Rules of thumb borrowed from kernel density estimation, such as Silverman's $s = 1.06\,\hat{\sigma}\, n^{-1/5}$ for a scalar input with sample standard deviation $\hat{\sigma}$, give a quick initial value, accurate for roughly Gaussian inputs but over-smoothing for multimodal or heavy-tailed ones. In d dimensions the suggested bandwidth decreases as $n^{-1/(d+4)}$, more slowly as d grows: more data are needed to fill a higher-dimensional space at the same resolution, an aspect of the curse of dimensionality discussed in Section 8.

4 Locally Weighted Regression

4.1 Local Linear Fits

The Nadaraya–Watson estimator fits, in effect, a constant in each neighborhood. This works when the regression function is nearly flat at the scale of the bandwidth, but introduces a systematic bias where it has a non-negligible slope. The effect is strongest at the boundaries of the data, where the neighborhood is one-sided and the average pulls the estimate toward the interior.

[1] In *leave-one-out* cross-validation each training item is used in turn as a validation set of size one: the estimator is computed on the other $n - 1$ items and evaluated on the one left out, and the average of the n errors estimates the expected risk. It suits kernel regression well. Since the estimator has no training phase, leaving out $(\mathbf{x}_i, t_i)$ only means dropping the i-th term from the sums of Equation (6), so that all n predictions cost as much as a single evaluation of the estimator on the training set, while each of them uses almost all the data.

Locally weighted regression (LOESS) fits instead a linear model at each query point $\mathbf{x}$, minimizing the squared error weighted by proximity to $\mathbf{x}$,

$$E_{\mathbf{x}}(\overline{\mathbf{w}}) \;=\; \sum_{i=1}^{n} \psi_i(\mathbf{x}) \big(\overline{\mathbf{w}}^T \overline{\mathbf{x}}_i - t_i\big)^2, \qquad \psi_i(\mathbf{x}) \;=\; \kappa_s(\mathbf{x} - \mathbf{x}_i), \tag{9}$$

so that the fitted model is determined mainly by the items near $\mathbf{x}$. With the weights collected in the diagonal matrix $\boldsymbol{\Psi}(\mathbf{x})$, with entries $\psi_i(\mathbf{x})$, this is a weighted least-squares problem, whose solution generalizes the normal equations of the notes on linear regression:

$$\overline{\mathbf{w}}^*(\mathbf{x}) \;=\; \big(\overline{\mathbf{X}}^T \boldsymbol{\Psi}(\mathbf{x})\, \overline{\mathbf{X}}\big)^{-1} \overline{\mathbf{X}}^T \boldsymbol{\Psi}(\mathbf{x})\, \mathbf{t}, \qquad h(\mathbf{x}) \;=\; \overline{\mathbf{w}}^*(\mathbf{x})^T \overline{\mathbf{x}}. \tag{10}$$

The parameters change with the query, and each prediction requires the solution of a separate linear system: this is the price of the reduced bias with respect to Nadaraya–Watson.

Local polynomial fits of higher degree reduce the bias further, but have more coefficients, $d + 1 + \dfrac{d(d+1)}{2}$ for a quadratic in d dimensions, and fitting them reliably requires neighborhoods containing at least as many well-spread points: a larger degree forces a larger bandwidth, which partially offsets the reduction in bias. The standard kernel in LOESS implementations is the tricube kernel, $\kappa_s(u) = \left(1 - \left(\dfrac{|u|}{s}\right)^3\right)^3$ for $|u| < s$ and 0 otherwise, and the bandwidth is set by cross-validation.

4.2 Local Logistic Regression

The same idea applies to classification, replacing the squared error with the cross-entropy of logistic regression, discussed in the classification notes. At a query $\mathbf{x}$, the parameters minimize the weighted cross-entropy

$$E_{\mathbf{x}}(\overline{\mathbf{w}}) \;=\; -\sum_{i=1}^{n} \kappa_s(\mathbf{x} - \mathbf{x}_i) \big(t_i \log p_i + (1 - t_i) \log(1 - p_i)\big), \qquad p_i \;=\; \sigma(\overline{\mathbf{w}}^T \overline{\mathbf{x}}_i), \tag{11}$$

and the decision boundary adapts to the geometry of the data near $\mathbf{x}$, instead of being a single global hyperplane. Where the boundary is approximately linear at the scale of the bandwidth, the local model recovers it accurately; overall, it produces a piecewise-linear approximation of a curved boundary, without specifying a nonlinear form in advance. The method is useful in low dimension, typically $d \leq 3$, when the boundary is genuinely nonlinear: in high dimension the neighborhoods become sparse and the local fits unreliable, and kernel support vector machines or neural networks, discussed later in the course, are preferable.

5 Gaussian Processes

5.1 From a Prior on Parameters to a Prior on Functions

In Bayesian linear regression the prior is placed on the parameters, but what it really expresses is a belief about functions. Each $\mathbf{w}$ drawn from $\mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$ gives a function $f(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$, so that drawing a parameter vector is the same as drawing a function, and the prior on $\mathbf{w}$ induces a distribution over the functions that the model can represent. The parameters are only a convenient way to index these functions: predictions depend on $\mathbf{w}$ only through the values $f(\mathbf{x})$ it produces, and two parameter vectors giving the same function would be indistinguishable. This suggests reasoning directly in terms of functions, and asking what the prior on $\mathbf{w}$ says about them.

The induced distribution is easy to describe at any finite set of points $\mathbf{x}_1, \ldots, \mathbf{x}_m$. The vector of values $\mathbf{f} = (f(\mathbf{x}_1), \ldots, f(\mathbf{x}_m))^T = \boldsymbol{\Phi}\mathbf{w}$, where $\boldsymbol{\Phi}$ is the design matrix of the m points, is a

linear transformation of a Gaussian vector, hence Gaussian. Its mean is $\mathbb{E}[\boldsymbol{\Phi}\mathbf{w}] = \boldsymbol{\Phi}\,\mathbb{E}[\mathbf{w}] = \mathbf{0}$, and its covariance is $\boldsymbol{\Phi}\,\mathrm{cov}(\mathbf{w})\,\boldsymbol{\Phi}^T = \alpha^{-1}\boldsymbol{\Phi}\boldsymbol{\Phi}^T$, that is,

$$\mathrm{cov}\big(f(\mathbf{x}_i), f(\mathbf{x}_j)\big) \;=\; \kappa(\mathbf{x}_i, \mathbf{x}_j) \;=\; \frac{1}{\alpha}\,\boldsymbol{\phi}(\mathbf{x}_i)^T\boldsymbol{\phi}(\mathbf{x}_j). \tag{12}$$

The parameters have disappeared: the distribution over functions is entirely described by the function κ, which gives the covariance between the values of f at any two points. This kernel is, up to a constant, the one that defines the equivalent kernel of Section 2.

A **Gaussian process** takes this as a starting point. Instead of choosing basis functions and a prior on their weights, it specifies directly the covariance function κ, and with it a distribution over functions, possibly much richer than any finite set of basis functions could produce. The object of inference changes accordingly. A **random function** is a collection of random variables $f(\mathbf{x})$, one for each input $\mathbf{x}$: at each point the value of the function is uncertain, and the values at different points are, in general, dependent. A distribution over functions specifies both aspects. The distribution of each $f(\mathbf{x})$ expresses how uncertain we are about the value of the function at $\mathbf{x}$. The dependence between $f(\mathbf{x})$ and $f(\mathbf{x}')$ expresses how much observing the function at one point tells us about it at the other. This dependence is what makes learning possible: if the values at different points were independent, observing f at the training inputs would say nothing about its value anywhere else.

This view has several implications. The prior expresses assumptions on the *shape* of the function, such as its smoothness, its typical amplitude, the scale over which it varies, or its periodicity, rather than on the values of parameters that are hard to interpret. Learning is again Bayesian inference: observing the training data turns the prior over functions into a posterior over functions, concentrated on those compatible with the observations. Prediction at a new input is the distribution of the value of the posterior function there, which comes with a measure of its uncertainty. And the complexity of the model is no longer fixed in advance by a number of basis functions: as shown below, the method is *nonparametric*, and the predictions are built from the training data themselves.

A function, however, is an infinite-dimensional object, and a distribution over functions cannot be handled as a distribution over a vector. What saves the approach is that every computation involves the values of f at finitely many points only: the n training inputs, at which the function is observed, and the query points, at which predictions are required. The values at all the other points never enter the computation. It is therefore enough to know, for every finite set of points, the joint distribution of the values of f there, a distribution on a vector of finite dimension. These **finite-dimensional distributions** cannot be chosen independently of each other. They must be *consistent*: the distribution of the values at a subset of points must be the marginal of the one at the whole set, so that adding a point to the set, and integrating it out, leaves the distribution of the other values unchanged. A classical result, the Kolmogorov extension theorem, guarantees that a random function with exactly these finite-dimensional distributions then exists. In practice, a Gaussian process is never manipulated as a whole: its distribution is evaluated on the finite set of points relevant to the problem, and the remaining infinitely many values are implicitly integrated out. The cost of inference depends on the number of points considered, not on the dimension of the space of functions.

Gaussian distributions are the natural choice, for three reasons. Marginals of a multivariate Gaussian are Gaussian, so that consistency is automatic. Conditionals of a multivariate Gaussian are Gaussian too, with mean and covariance in closed form: conditioning on the observed values, that is, Bayesian inference, is exact. Finally, a Gaussian is described by its mean and covariance, so that the whole distribution over functions is described by two functions, one giving the mean of each value and one giving the covariance of each pair of values.

The **mean function** $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$ is the expected value of the function at $\mathbf{x}$ before observing the data, a baseline around which the functions of the prior fluctuate. Prior knowledge, such as a known trend, can be encoded in it, but it is usually set to zero, after centering the targets. As shown in Section 6, this is not a restriction: the posterior mean is pulled toward the observed targets, and learns their level and trend from the data.

The covariance function, or kernel, is

$$\kappa(\mathbf{x}, \mathbf{x}') \;=\; \operatorname{cov}\big(f(\mathbf{x}), f(\mathbf{x}')\big) \;=\; \mathbb{E}\left[\big(f(\mathbf{x}) - m(\mathbf{x})\big)\big(f(\mathbf{x}') - m(\mathbf{x}')\big)\right], \tag{13}$$

and carries the actual assumptions on the function. Its value $\kappa(\mathbf{x}, \mathbf{x})$ on the diagonal is the variance of $f(\mathbf{x})$, which sets the typical amplitude of the fluctuations around the mean at $\mathbf{x}$. Its value on a pair of distinct points, compared with the two variances, gives the correlation $\kappa(\mathbf{x}, \mathbf{x}')/\sqrt{\kappa(\mathbf{x}, \mathbf{x})\,\kappa(\mathbf{x}', \mathbf{x}')}$ between the values there. If nearby points have values strongly correlated, the functions of the prior vary little over short distances and are smooth. How fast the correlation decays with distance sets the scale over which the functions vary. A correlation that repeats at regular distances produces periodic functions. Not every function of two arguments can play this role. For every finite set of points, the matrix of the values of κ must be a valid covariance matrix, which, as discussed in the next section, requires κ to be symmetric and positive definite.

The kernel of Bayesian linear regression is a special case, and also a degenerate one. Its Gram matrices have rank at most equal to the number of basis functions, since they are products $\alpha^{-1}\boldsymbol{\Phi}\boldsymbol{\Phi}^T$, so that the functions of the prior are confined to the finite-dimensional space spanned by the basis functions. Kernels such as the RBF kernel introduced below have Gram matrices of full rank for any set of distinct points. They correspond to models with infinitely many basis functions, which cannot be handled through their weights, but only through the kernel.

5.2 Definition

A stochastic process on a domain $\mathcal{X}$, typically $\mathbb{R}^d$, is a collection of random variables $\{f(\mathbf{x}) : \mathbf{x} \in \mathcal{X}\}$, which can be seen as a random function on $\mathcal{X}$. It is a Gaussian process, written $\mathcal{GP}(m, \kappa)$, if, for every finite set of points $\mathbf{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_m)$, the values $f(\mathbf{x}_1), \ldots, f(\mathbf{x}_m)$ are jointly Gaussian,[2]

$$\big(f(\mathbf{x}_1), \ldots, f(\mathbf{x}_m)\big) \;\sim\; \mathcal{N}(\boldsymbol{\mu}_\mathbf{X}, \mathbf{G}_\mathbf{X}), \qquad \big[\boldsymbol{\mu}_\mathbf{X}\big]_i \;=\; m(\mathbf{x}_i), \qquad \big[\mathbf{G}_\mathbf{X}\big]_{ij} \;=\; \kappa(\mathbf{x}_i, \mathbf{x}_j). \tag{14}$$

Drawing a function from the process, on a finite grid of points, thus amounts to drawing a vector from this Gaussian. The mean function is usually set to zero, a choice that, as shown in Section 6, involves no loss of generality: the prior is then entirely specified by the kernel, and a large $\kappa(\mathbf{x}, \mathbf{x}')$ means that $f(\mathbf{x})$ and $f(\mathbf{x}')$ tend to be similar.

Not every function can be a kernel: $\mathbf{G}_\mathbf{X}$, the Gram matrix of κ on $\mathbf{X}$, must be a valid covariance matrix for every choice of the points. This holds when κ is symmetric and positive definite, that is, when

$$\sum_{i=1}^{n}\sum_{j=1}^{n} c_i c_j\, \kappa(\mathbf{x}_i, \mathbf{x}_j) \;>\; 0 \tag{15}$$

for every set of distinct points and every $c_1, \ldots, c_n$ not all zero, so that the Gram matrices have positive eigenvalues. Many positive definite kernels are known, and new ones can be built from them, since sums and products of positive definite kernels are positive definite.

5.3 Kernels

Through the kernel, the prior expresses beliefs on the smoothness, length scale and amplitude of the function. One of the most widely used kernels is the RBF kernel

$$\kappa(\mathbf{x}, \mathbf{x}') \;=\; \sigma_f^2\, e^{-\frac{||\mathbf{x}-\mathbf{x}'||^2}{2\ell^2}}, \tag{16}$$

which makes the values of f at nearby points strongly correlated, and those at distant points nearly independent. Functions sampled from this prior are smooth: the signal variance σ_f^2 sets their amplitude, and the length scale ℓ the distance over which they vary significantly, so that a large ℓ gives slowly varying functions and a small ℓ rapid fluctuations (Figure 4).

[2] All the results below apply unchanged when $\boldsymbol{\phi}(\mathbf{x})$ is used in place of $\mathbf{x}$.

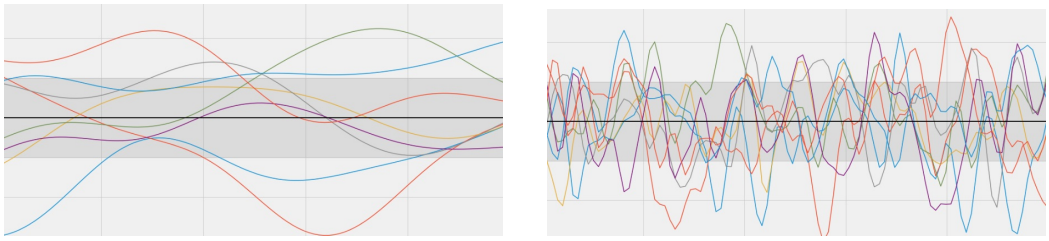


Figure 4: Functions sampled from a Gaussian process with RBF kernel, for a large (left) and a small (right) length scale ℓ. A smaller ℓ gives faster variations, while the functions remain smooth.

A single length scale imposes the same smoothness in all input directions. If f depends strongly on x_1 and barely on x_2, two points close in x_1 but distant in x_2 receive a low covariance, although their function values are similar. **Automatic relevance determination** (ARD) kernels use instead a length scale for each feature,

$$\kappa(\mathbf{x}, \mathbf{x}') \;=\; \sigma_f^2 \exp\!\Big(-\frac{1}{2}\sum_{k=1}^{d} \frac{(x_k - x_k')^2}{\ell_k^2}\Big). \tag{17}$$

A large ℓ_k makes the kernel insensitive to the k-th feature, effectively removing it from the model: when the length scales are learned from the data, as in Section 8, this performs feature selection with no explicit selection step.

6 Gaussian Process Regression

6.1 Noise-Free Observations

Consider first the case in which each target is the exact value of the function, $t_i = f(\mathbf{x}_i)$, and the predictions are needed at the test points $\mathbf{Z} = (\mathbf{z}_1, \ldots, \mathbf{z}_r)$. Before observing the data, the prior allows a wide variety of functions, constrained only by the smoothness encoded by the kernel (Figure 5).

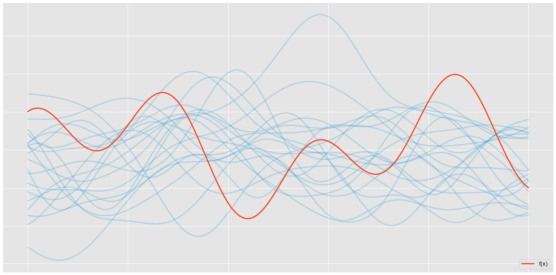


Figure 5: The unknown function f (red) and functions sampled from a Gaussian process prior (blue).

Prediction rests on a single fact: by definition of Gaussian process, the values at the training points and at the test points are jointly Gaussian,

$$\begin{pmatrix} f(\mathbf{X}) \\ f(\mathbf{Z}) \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_{\mathbf{X}} \\ \boldsymbol{\mu}_{\mathbf{Z}} \end{pmatrix}, \begin{pmatrix} \mathrm{G}_{\mathbf{X}} & \mathrm{G}_{\mathbf{Z},\mathbf{X}}^{T} \\ \mathrm{G}_{\mathbf{Z},\mathbf{X}} & \mathrm{G}_{\mathbf{Z}} \end{pmatrix} \right), \qquad \big[\mathbf{G}_{\mathbf{Z},\mathbf{X}}\big]_{ij} \;=\; \kappa(\mathbf{z}_i, \mathbf{x}_j). \tag{18}$$

The diagonal blocks are the covariances among training values and among test values; the cross-covariance $\mathbf{G}_{\mathbf{Z},\mathbf{X}}$ is what links them, and through it the observations at $\mathbf{X}$ inform the predictions at $\mathbf{Z}$. The posterior on $f(\mathbf{Z})$ is the distribution of $f(\mathbf{Z})$ conditioned on $f(\mathbf{X}) = \mathbf{t}$, obtained with the standard formulas for Gaussian conditioning.

Conditioning a multivariate Gaussian

Let $\mathbf{v} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ be partitioned as $\mathbf{v} = (\mathbf{v}_A, \mathbf{v}_B)$, with

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_A \\ \boldsymbol{\mu}_B \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_A & \boldsymbol{\Sigma}_{AB} \\ \boldsymbol{\Sigma}_{AB}^T & \boldsymbol{\Sigma}_B \end{pmatrix}.$$

The marginals are $\mathcal{N}(\boldsymbol{\mu}_A, \boldsymbol{\Sigma}_A)$ and $\mathcal{N}(\boldsymbol{\mu}_B, \boldsymbol{\Sigma}_B)$, and the distribution of $\mathbf{v}_A$ conditioned on $\mathbf{v}_B$ is Gaussian, with

$$\boldsymbol{\mu}_{A|B} = \boldsymbol{\mu}_A + \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_B^{-1}(\mathbf{v}_B - \boldsymbol{\mu}_B), \qquad \boldsymbol{\Sigma}_{A|B} = \boldsymbol{\Sigma}_A - \boldsymbol{\Sigma}_{AB}\boldsymbol{\Sigma}_B^{-1}\boldsymbol{\Sigma}_{AB}^T.$$

The conditional mean moves away from $\boldsymbol{\mu}_A$ by an amount proportional to the deviation of $\mathbf{v}_B$ from its mean, and larger when A and B are strongly correlated. The conditional covariance does not depend on the observed value: observing $\mathbf{v}_B$ reduces the uncertainty on $\mathbf{v}_A$ by a fixed amount, whatever the value observed.

With $\mathbf{v}_A = f(\mathbf{Z})$ and $\mathbf{v}_B = f(\mathbf{X}) = \mathbf{t}$, the posterior of $f(\mathbf{Z})$ is Gaussian, with

$$\boldsymbol{\mu}_p = \boldsymbol{\mu}_\mathbf{Z} + \mathbf{G}_{\mathbf{Z},\mathbf{X}}\mathbf{G}_\mathbf{X}^{-1}(\mathbf{t} - \boldsymbol{\mu}_\mathbf{X}), \qquad \boldsymbol{\Sigma}_p = \mathbf{G}_\mathbf{Z} - \mathbf{G}_{\mathbf{Z},\mathbf{X}}\mathbf{G}_\mathbf{X}^{-1}\mathbf{G}_{\mathbf{Z},\mathbf{X}}^T. \tag{19}$$

Both formulas have a simple reading. The posterior mean is the prior mean, corrected according to how far the observed targets are from the prior mean at the training points; the matrix $\mathbf{G}_{\mathbf{Z},\mathbf{X}}\mathbf{G}_\mathbf{X}^{-1}$ determines how strongly each observation affects each test point. A test point far from all training points has small covariances with them, and its mean stays close to the prior one; a test point close to $\mathbf{x}_i$ has its mean pulled toward t_i. The posterior covariance is the prior one minus a positive semidefinite term: observing data can only reduce uncertainty, little far from the training points and much near them.

With zero prior mean, the case considered from now on, the posterior mean is $\mathbf{G}_{\mathbf{Z},\mathbf{X}}\mathbf{G}_\mathbf{X}^{-1}\mathbf{t}$, which is not zero whenever the targets are not: the level and trend of the function are learned from the data. A non-zero mean function can always be reintroduced afterwards, since Equation (19) only adds to this mean the closed-form term $\boldsymbol{\mu}_\mathbf{Z} - \mathbf{G}_{\mathbf{Z},\mathbf{X}}\mathbf{G}_\mathbf{X}^{-1}\boldsymbol{\mu}_\mathbf{X}$. For a single test point $\mathbf{z}$, writing $\mathbf{g}_\mathbf{z} = (\kappa(\mathbf{z}, \mathbf{x}_1), \ldots, \kappa(\mathbf{z}, \mathbf{x}_n))^T$ for its covariances with the training points, the prediction is

$$\mu_p(\mathbf{z}) = \mathbf{g}_\mathbf{z}^T\mathbf{G}_\mathbf{X}^{-1}\mathbf{t}, \qquad \sigma_p^2(\mathbf{z}) = \kappa(\mathbf{z}, \mathbf{z}) - \mathbf{g}_\mathbf{z}^T\mathbf{G}_\mathbf{X}^{-1}\mathbf{g}_\mathbf{z}. \tag{20}$$

The mean is again a weighted sum of the training targets, with weights $\mathbf{g}_\mathbf{z}^T\mathbf{G}_\mathbf{X}^{-1}$ playing the role of the equivalent kernel of Section 2. At a training point $\mathbf{x}_i$, $\mathbf{g}_{\mathbf{x}_i}$ is the i-th column of $\mathbf{G}_\mathbf{X}$, so that $\mu_p(\mathbf{x}_i) = t_i$ and $\sigma_p^2(\mathbf{x}_i) = 0$: in the noise-free case the posterior interpolates the data, and every function sampled from it passes through the observed values, while the uncertainty grows between and beyond them (Figures 6 and 7).

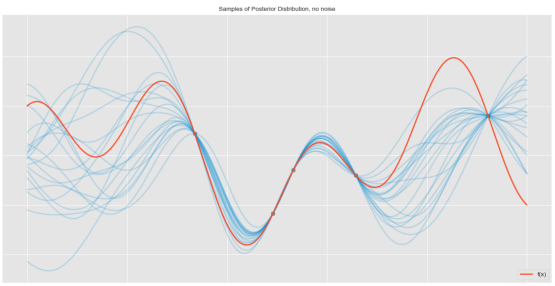


Figure 6: Functions sampled from the posterior, after observing f (red) at 5 points: in the noise-free case, every sample passes through the observed values.

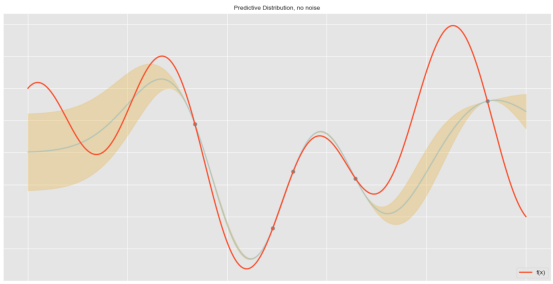


Figure 7: Predictive mean (blue) and one-standard-deviation band (yellow) in the noise-free case: the mean interpolates the observed values, and the uncertainty grows away from them.

6.2 Noisy Observations

Exact observations are rarely realistic. As in the probabilistic model of linear regression, each target can be treated as a noisy measurement of the function value,

$$t_i \;=\; f(\mathbf{x}_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma_n^2), \tag{21}$$

with noise independent across items and of f (Figure 8). The targets are then jointly Gaussian with the test values, as before, with a single change: the covariance of the targets is that of the function values plus the noise variance on the diagonal,

$$\hat{\mathbf{\Sigma}}_{\mathbf{X}} \;=\; \mathbf{G}_{\mathbf{X}} + \sigma_n^2 \mathbf{I}, \tag{22}$$

while the cross-covariance $\mathbf{G}_{\mathbf{Z},\mathbf{X}}$ is unchanged, since the noise affects the observations, not the function.

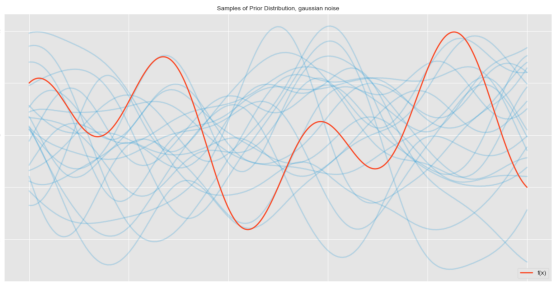


Figure 8: Observations with noise: the observed values no longer lie on the function.

The posterior is given by the same formulas, with $\mathbf{G}_{\mathbf{X}}$ replaced by $\hat{\mathbf{\Sigma}}_{\mathbf{X}}$; for a single test point, with zero mean,

$$\mu_p(\mathbf{z}) \;=\; \mathbf{g}_{\mathbf{z}}^T \hat{\mathbf{\Sigma}}_{\mathbf{X}}^{-1} \mathbf{t}, \qquad \sigma_p^2(\mathbf{z}) \;=\; \kappa(\mathbf{z},\mathbf{z}) - \mathbf{g}_{\mathbf{z}}^T \hat{\mathbf{\Sigma}}_{\mathbf{X}}^{-1} \mathbf{g}_{\mathbf{z}}, \tag{23}$$

to which σ_n^2 is added when the prediction concerns a new noisy target rather than the function value. The term $\sigma_n^2 \mathbf{I}$ makes the matrix to be inverted larger, and the corrections smaller: the mean is pulled less toward the observed targets, and the variance at the observed points no longer vanishes. The posterior smooths the data instead of interpolating them, to a degree controlled by σ_n^2 (Figures 9 and 10). With the kernel $\kappa(\mathbf{x},\mathbf{x}') = \alpha^{-1}\boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{x}')$ and $\sigma_n^2 = 1/\beta$, these are exactly the predictive mean and variance of Bayesian linear regression, now obtained without ever introducing the weights.

7 Kernels Across Methods

A kernel measures the similarity of two inputs in a way relevant to the prediction, but its role differs across methods. In kernel regression, $\kappa_s(\mathbf{x} - \mathbf{x}_i)$ is a *weighting function*, applied to the difference

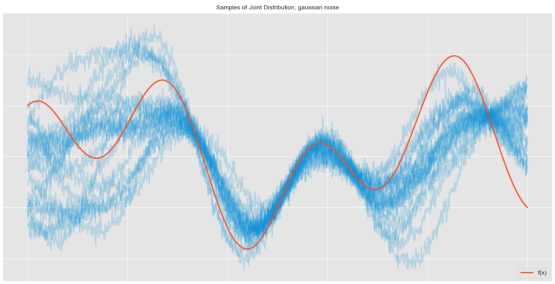


Figure 9: Functions sampled from the posterior with noisy observations: they no longer interpolate the observed values, but smooth through them.

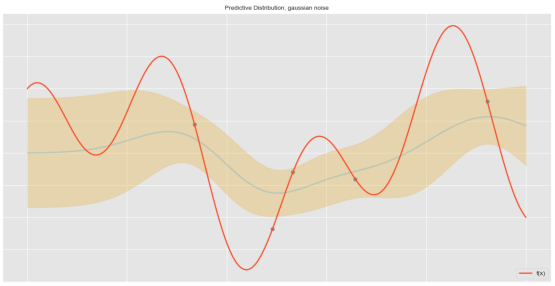


Figure 10: Predictive mean and one-standard-deviation band with noisy observations: the mean does not pass through the data, and the band remains wider at the observed points than in the noise-free case.

between the query and a training input: its value is the influence of the observation at $\mathbf{x}_i$ on the prediction at $\mathbf{x}$, and the kernel is a procedural tool rather than a probability model; it only needs to be non-negative, and to integrate to one for the derivation of Section 3. In Gaussian processes, $\kappa(\mathbf{x}, \mathbf{x}')$ is the *covariance* of $f(\mathbf{x})$ and $f(\mathbf{x}')$: a large value means that knowing one of them is informative about the other, and the kernel encodes prior beliefs on the smoothness and structure of the function. In support vector machines, discussed later in the course, $\kappa(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x}')$ is an *inner product* in a possibly infinite-dimensional feature space, which the kernel allows to use without computing $\boldsymbol{\phi}$ explicitly.

In the last two cases, the kernel is a symmetric positive definite function on $\mathcal{X} \times \mathcal{X}$, and its Gram matrices have positive eigenvalues, as required both of a covariance matrix and of a matrix of inner products. The same mathematical object is thus used as a probabilistic covariance, as an inner product, and, in a looser form, as a weight.

8 Choosing the Kernel Parameters

The predictions of a Gaussian process depend heavily on the kernel hyperparameters: for the RBF kernel with noise, the signal variance σ_f^2, the length scale ℓ and the noise variance σ_n^2, whose different values produce qualitatively different predictions (Figure 11). Rather than by hand or by cross-validation, they can be chosen, as in the evidence framework of the notes on linear regression, by maximizing the **marginal likelihood** of the training targets, $p(\mathbf{t} \mid \mathbf{X}; \boldsymbol{\theta}) = \mathcal{N}(\mathbf{t} \mid \mathbf{0}, \hat{\boldsymbol{\Sigma}}_{\mathbf{X}})$, with $\boldsymbol{\theta}$ the hyperparameters:

$$\log p(\mathbf{t} \mid \mathbf{X}; \boldsymbol{\theta}) \;=\; -\frac{1}{2}\mathbf{t}^T \hat{\boldsymbol{\Sigma}}_{\mathbf{X}}^{-1} \mathbf{t} - \frac{1}{2}\log\left|\hat{\boldsymbol{\Sigma}}_{\mathbf{X}}\right| - \frac{n}{2}\log(2\pi). \tag{24}$$

The first term rewards the fit to the data; the second is an Occam penalty, larger for flexible models, such as those with a short length scale, which spread their probability over many possible target vectors. The maximization, by gradient methods, balances the two automatically.

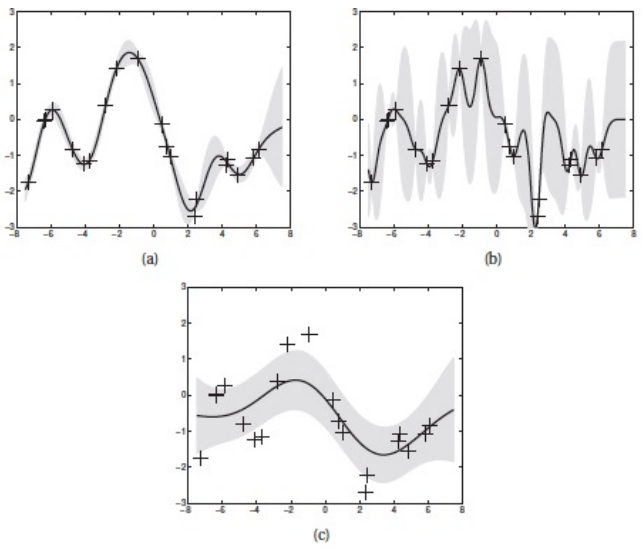


Figure 11: Predictions of a Gaussian process for $(\ell, \sigma_f, \sigma_n)$ equal to $(1, 1, 0.1)$, $(0.3, 1.08, 0.00005)$ and $(3.0, 1.16, 0.89)$: from a balanced fit, to a near-interpolation with a short length scale, to a heavy smoothing with a long length scale and a large noise variance.

All the local methods of these notes, Gaussian processes included, suffer from the **curse of dimensionality**: a ball of radius ϵ covers a fraction ϵ^d of the unit hypercube, so that in high dimension the neighborhood of a query contains almost no training data, and keeping a fixed number of neighbors requires neighborhoods so wide that local weights lose their meaning. ARD mitigates the problem when only a few features are relevant. Finally, Gaussian process regression requires the inversion of the $n \times n$ matrix $\hat{\mathbf{\Sigma}}_{\mathbf{X}}$, at a cost $O(n^3)$, which sparse approximations based on $m \ll n$ representative points reduce to $O(nm^2)$.